

[ACJC/2022/Q3]

- 3 In this task, you are **not** allowed to use the keyword `in` to determine whether a character exists in a string, or whether one string is a substring of another (e.g. `'G' in 'SINGAPORE'` or `'GAP' in 'SINGAPORE'`). You are also not allowed to check directly if two substrings longer than one character are equal (e.g. `'SINGAPORE'[0:3] == 'HELSINKI'[3:6]`).

The Boyer-Moore algorithm was designed to check if a string `s` is a substring of another string `t` (called the text).

Task 3.1

Write a function `dic(s)` that creates a dictionary whose keys are the characters of `s`, and the value of each key is the difference between the index of its last occurrence in `s`, from the largest index in `s`.

For example, `dic('anman')` creates a dictionary with the following keys and values:

Key	Value
'a'	1
'n'	0
'm'	2

[3]

Task 3.2

To determine whether `s` is a substring of `t`, we first compare the last letter of `s` with the character in `t` that has the corresponding index. The following shows two examples.

```
          *
t1: a n p a n m a n      t2: a n p m n m a n
s : a n m a n             s: a n m a n
          *                *
```

If they match, we move on to the second last letter, and so on, until there is a mismatch.

```
          *                *
t1: a n p a n m a n      t2: a n p m n m a n
s : a n m a n             s: a n m a n
          *                *
```

```
          *
t1: a n p a n m a n
s : a n m a n
          *
```

If we reach the first character of `s` and there is no mismatch, then `s` is a substring of `t`. If there is a mismatch (as illustrated in both examples above), then we shift `s` forward.

For `t1`, the mismatched character (`'p'`) does not appear in `s` at all. Therefore, we can shift `s` forward until the first character of `s` is after the mismatched character, and start checking again from the last character of `s`. In this case, we find that we can reach the first character of `s` without a mismatch, and therefore `s` is a substring of `t`.

```
          *
t1: a n p a n m a n
s :      a n m a n
          *
```

[ACJC/2022/Q3 Cont']

For t_2 , the mismatched character ('m') occurs in s . We therefore need to shift s so that the last occurrence of 'm' in ' s ' lines up with the 'm' in t_2 , and start checking again from the last character of s .

```
          *
t2: a n p m n m a n
s :   a n m a n
          *
```

In this case, there is a mismatch immediately, and the mismatched character ('m') occurs in s . We therefore need to shift s so that the last occurrence of 'm' in ' s ' lines up with this 'm' in t_2 .

```
          *
t2: a n p m n m a n
s :       a n m a n
          *
```

We start checking again from the last character of s .

There is an edge case in which the mismatch happens in which the mismatched character in t ('a') is a character which occurred to its right in s .

```
          * (mismatched character is the 'a')
t3: a n p a m a n m a n
s :   a n m a n
          *
```

(result of applying the shifting rule)

```
t3: a n p a m a n m a n
s : a n m a n
```

In this case, the shifting rule would shift s backwards, which is counterproductive. In this case, the best we can do is shift s one character forward and start checking again from the last character.

```
          *
t3: a n p a m a n m a n
s :       a n m a n
          *
```

The loop of checking and shifting continues until we shift s so that its last character is beyond the last character of t .

Write code to carry out this algorithm to check if s is a substring of t .

[10]

Test your code on the following test cases:

```
s = 'anman'
t1 = 'aman'
t2 = 'anpanman'
t3 = 'anpanpan'
t4 = 'anpamanman'
```

[1]

Download your program code and output for Task 3 as

TASK3_<your name>_<centre number>_<index number>.ipynb